

Accelerating MFEM applications using MAGMA solvers for packed symmetric matrices

MFEM workshop

September 22-25th, 2026

Arturo Vargas, Ahmad Abdelfattah, Natalie Beams

LLNL, ICL- UTK

High-order DG and finite-element methods generate many independent, small dense systems

- Typical examples:
 - Element-local mass matrices
 - Local quadrature and projection operations
 - Local block solves
- These systems are often:
 - Symmetric positive definite
 - Small, but numerous and may require repeated solves
 - Limited by memory movement rather than arithmetic compute
- **Key question:** How can we reduce memory footprint and data movement without sacrificing throughput?
 - At the start of this year, I started exploring packed triangular storage for an ALE hydro code from LLNL

Why packed storage?

- For an $n \times n$ symmetric matrix:
 - Packed storage: $n(n + 1)/2$ entries
 - Approximately 2× lower memory footprint
 - Reduced memory traffic for matrix operations
- While exploring this option I quickly realized no GPU vendor or MAGMA support for this format!
 - I developed my own kernels to bridge the gap (with the help of AI)
- Since April, we have been working with MAGMA team to develop optimized kernels for packed matrix structure

<https://github.com/mfem/mfem/tree/artv3/packed-triangular-format>

MFEM Operations and MAGMA Integration

- PR #5305 introduces packed-matrix data structures and operations
 - MassIntegrator::AssembleEATriangular
 - Matrix-Vec operations
 - Cholesky factorization, Cholesky-based inverse
 - Triangular solves
- MAGMA integration provides batched GPU kernels:
 - pptrf_batched: Cholesky factorization
 - pptrs_batched: Cholesky solve
 - ppinv_batched: Packed matrix inverse
 - Small-matrix optimized matrix-vector operations
 - **Current inverse support is limited to $n \leq 64$, and apply $n \leq 32$**

MAGMA Solvers for Packed Storage (batch $Ax = b$)

PATH 1

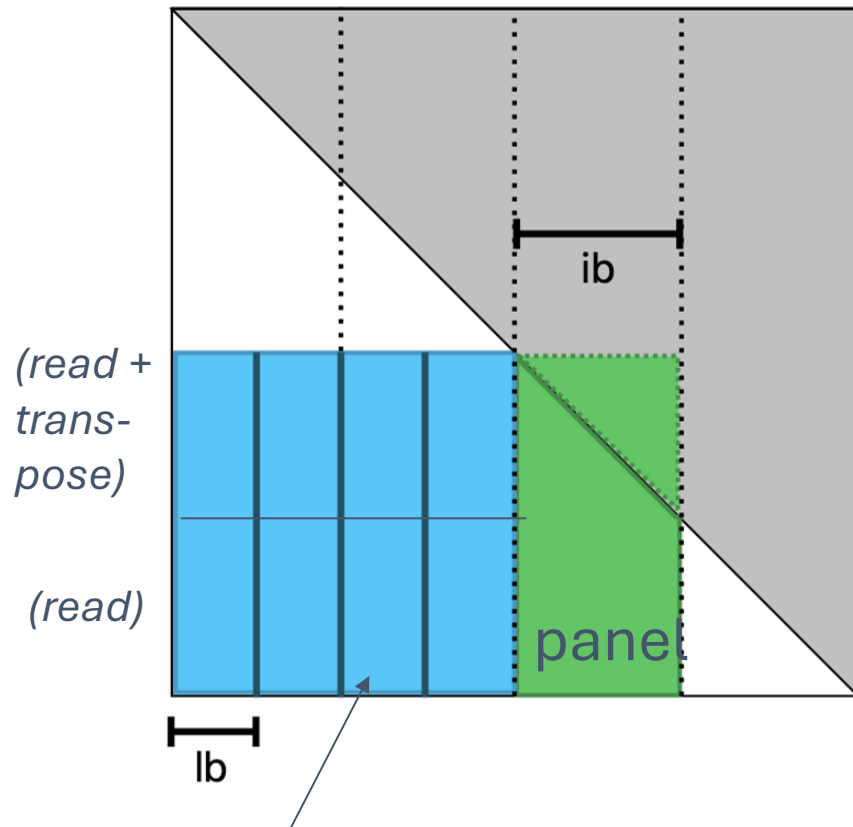
- 1) Factorization $A = LL^T$
- 2) Two tr. solves given L , b

PATH 2

- 1) Factorization $A = LL^T$
- 2) Invert A given L
- 3) Apply the inverse to b

- Path 1 is the standard way in LAPACK
- Path 2 could be faster depending on the number of solves/factorization

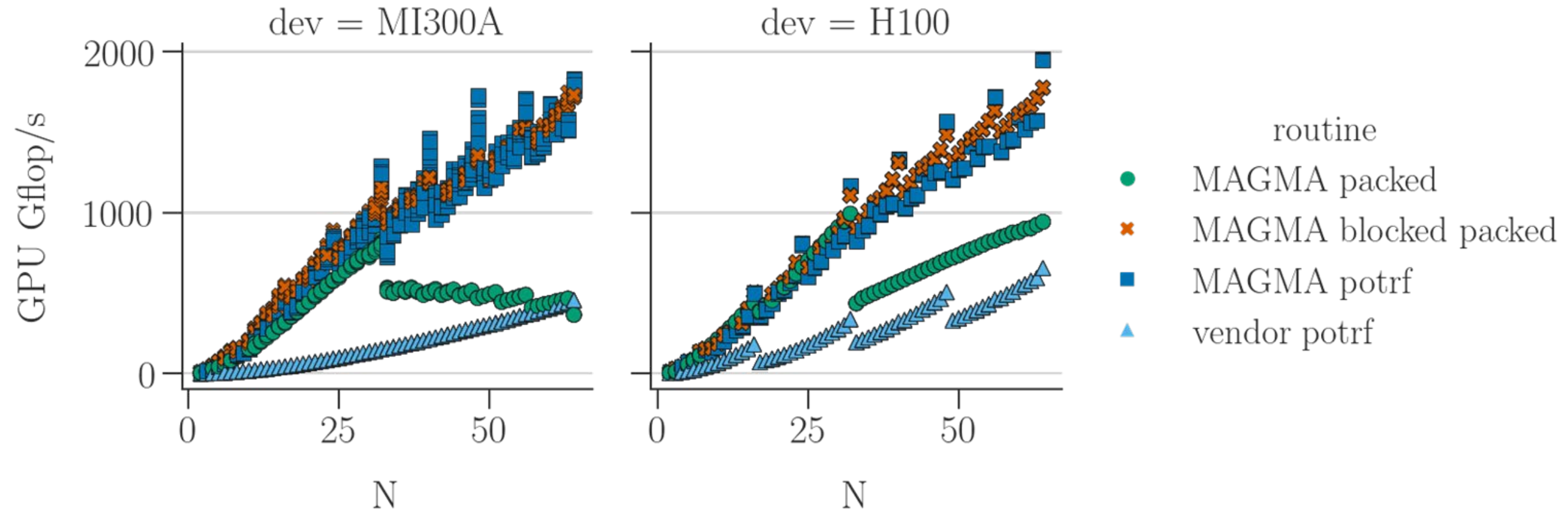
Batched Cholesky: MAGMA's strategy



- Left-looking Cholesky, each iteration includes
 - Panel update (GEMM/SYRK)
 - Panel factorization (POTF2, TRSM)
- One kernel call per iteration (fused update/factorize)
- Panel cached in shared memory between update/factorize stages
- Memory accesses support both packed and column-major layouts
- If the matrix is too small, the entire factorization is done in a single kernel call
 - Fully unblocked and unrolled code

Read/used in SYRK, GEMM panel updates

Packed Cholesky factorization: performance in FP64



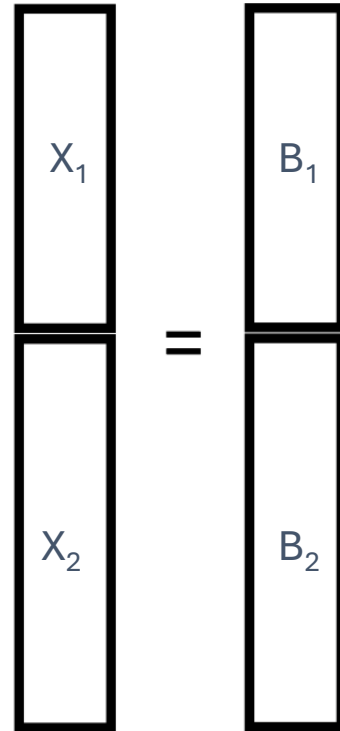
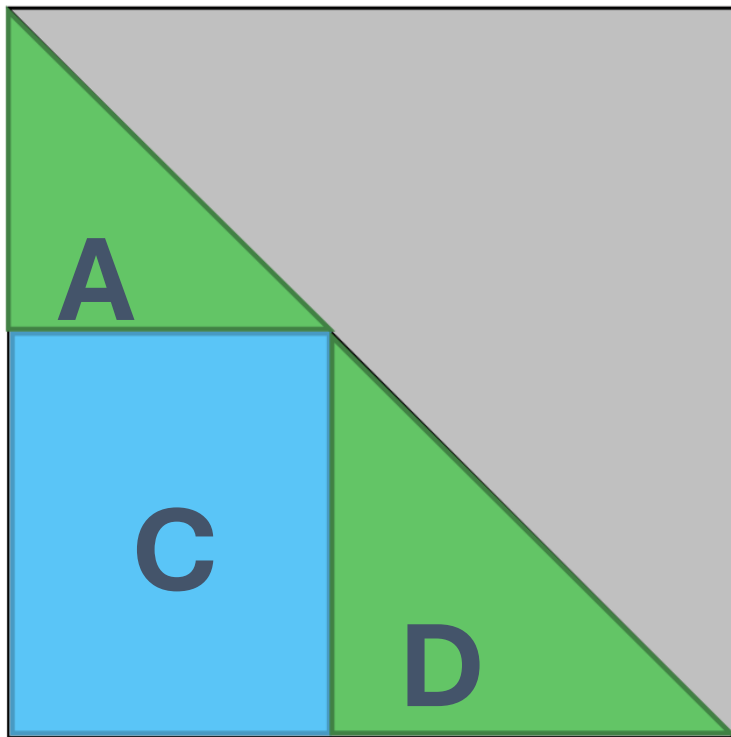
LLNL Tuolumne,
with ROCm 7.2.0

with CUDA 12.9

batch size: 65534

Triangular solve in MAGMA

- General strategy: recursive solve (TRSM + GEMM + TRSM)
- Memory accesses support both column-major and packed formats
- If the matrix is too small, we can fuse everything in one kernel

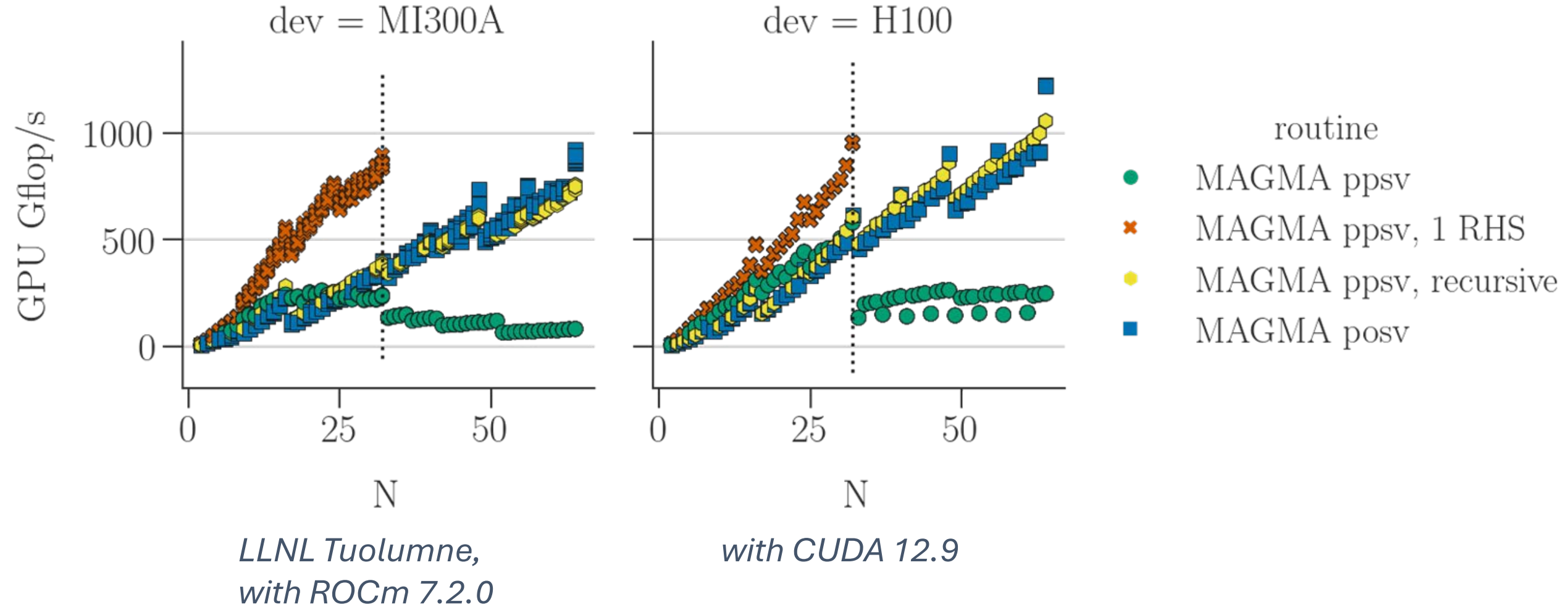


$$X_1 = A^{-1}B_1 \text{ (triangular solve)}$$

$$B_2 = B_2 - CX_1 \text{ (GEMM)}$$

$$X_2 = D^{-1}B_2 \text{ (triangular solve)}$$

Packed triangular solve: performance



batch size: 65534

Inversion-based solver

1) Batch Matrix Inversion

- a) Reuse the factorization routine
- b) Triangular solve with an Identity matrix
- c) High performance implementation for sizes up to 32x32
- d) [WIP] support for larger sizes (might need a workspace)

2) Batch Inverse-apply

- a) BLAS or BLAS-like kernels: $X = A^{-1} B$
- b) Single RHS: Hermitian packed matrix-vector product (HPMV)
- c) Multiple RHS: Packed version of the HEMM kernel
- d) There is a catch:
 - i) We need to do an in-place update

Direct solve vs. Inverse Apply?

Well, it depends:

- No. solve rounds (how many times the factors/inverse are being reused)
- No. RHS per linear system
- No. linear systems (batch size)
- Matrix size, precision, GPU, ... etc

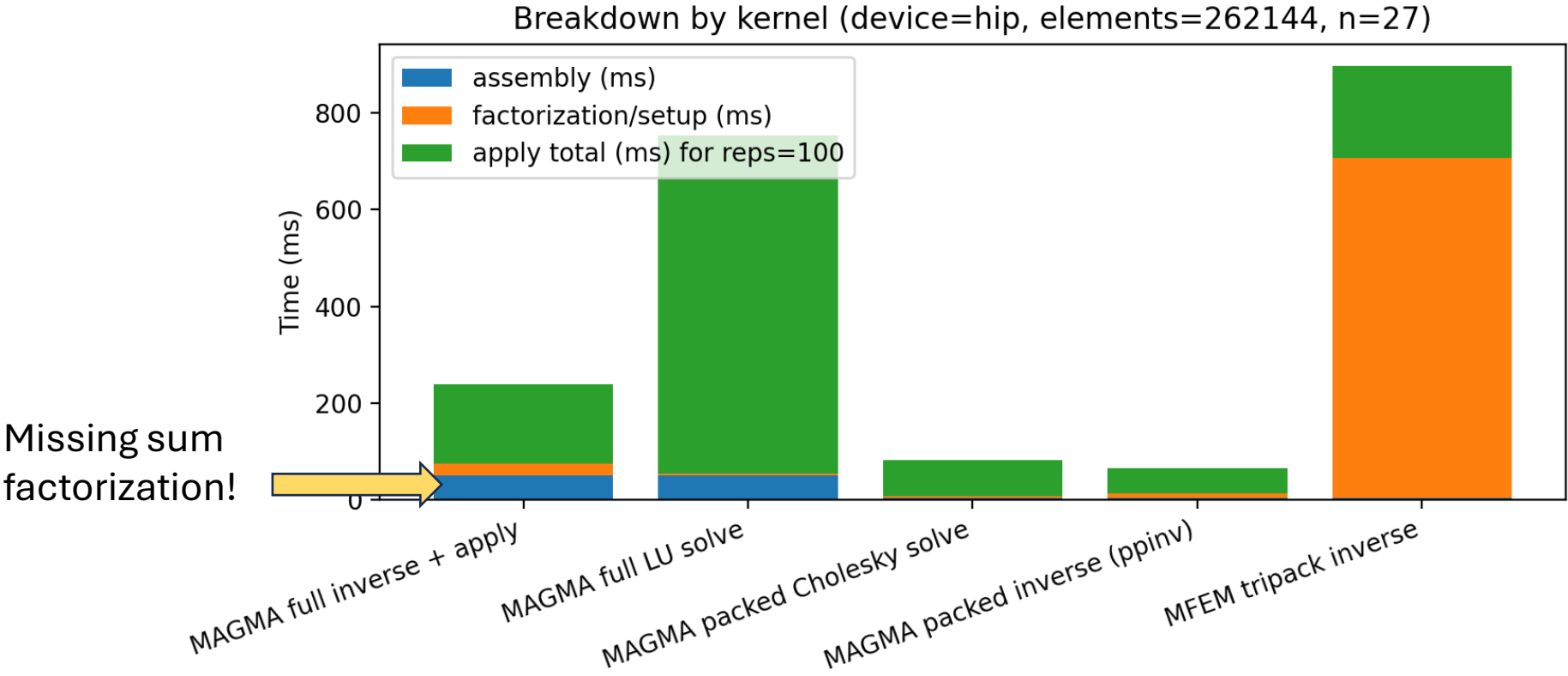
Evaluation objectives

We measure the impact of packed storage and MAGMA kernels on:

- Memory footprint
- Kernel throughput
- Factorization and solve time
- End-to-end application performance
- Test cases
 - MFEM benchmark and mini-app
 - MARBL benchmark problem

MAGMA batched matrix driver

MFEM's MAGMA backend is not SPD aware

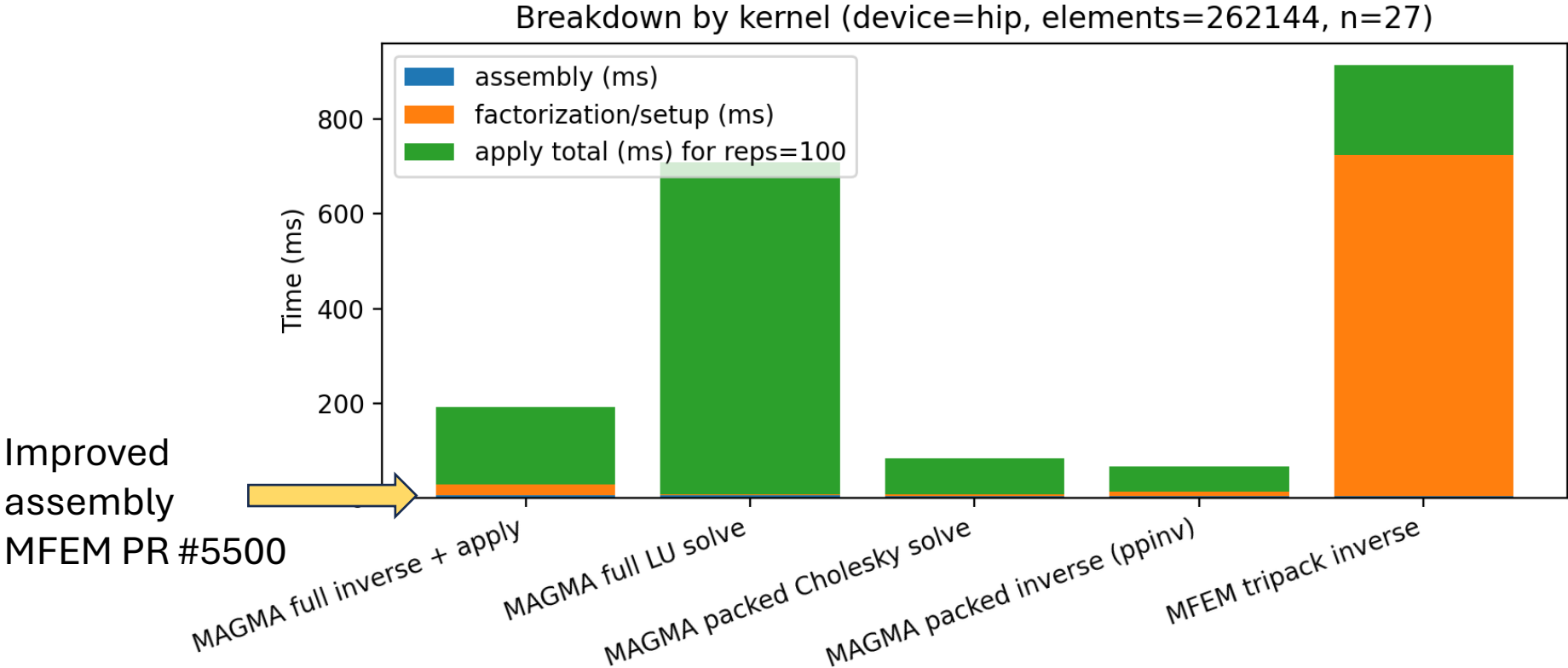


ROCM 7.2.0, MI300

Assembly for packed upper triangular matrix is optimized using sum factorization

MAGMA batched matrix driver

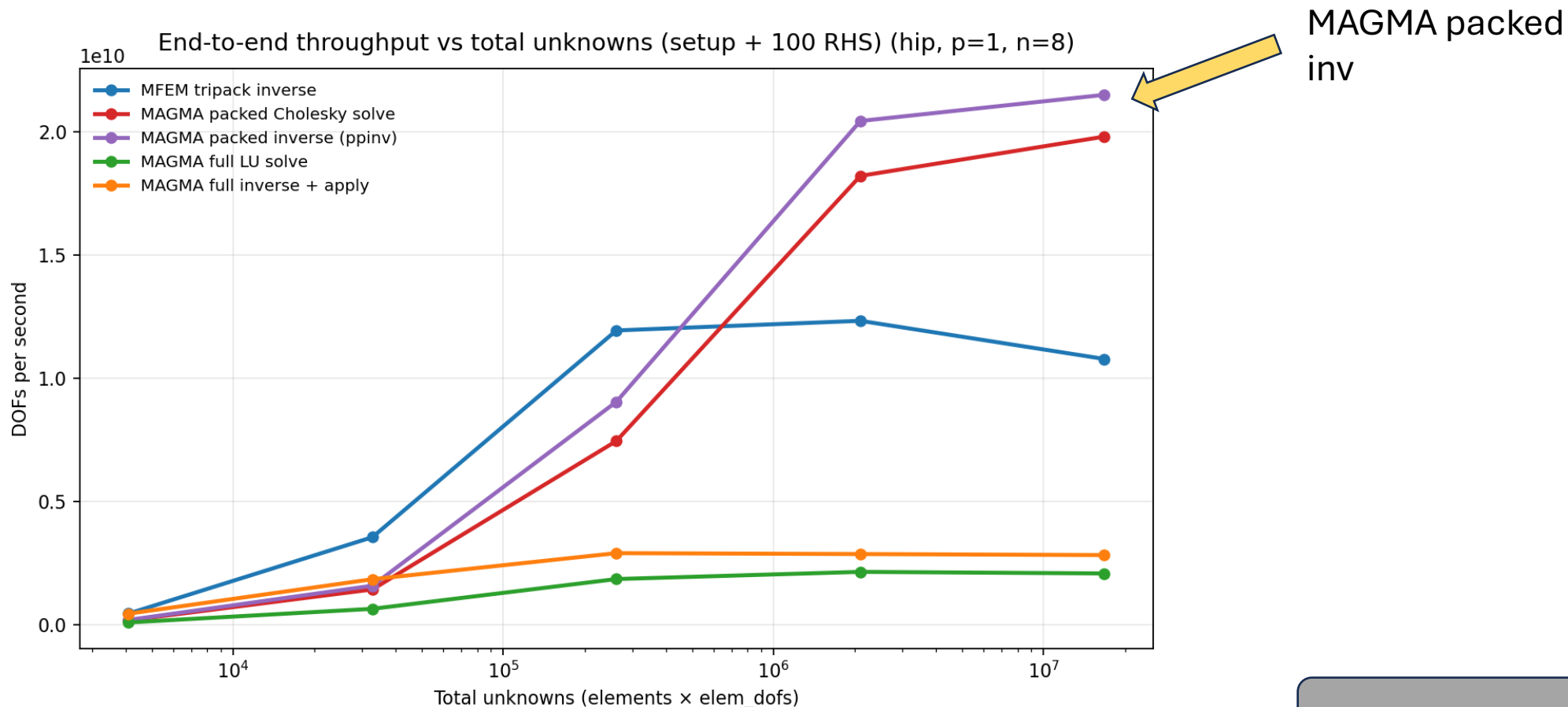
MFEM's MAGMA backend is not SPD aware



ROCM 7.2.0, MI300

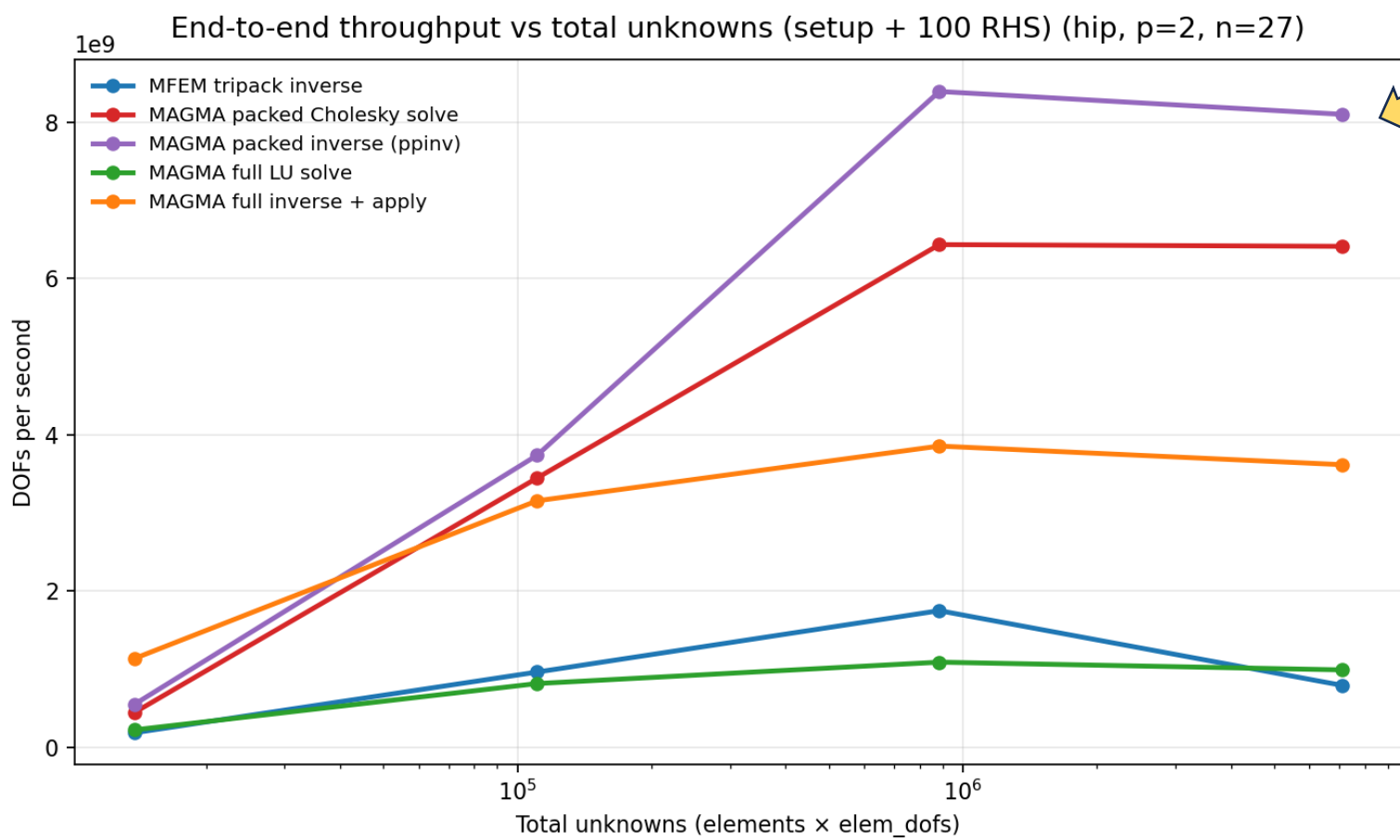
Packed Assembly is still ~1.17x faster in this case

MAGMA end-to-end throughput



ROCM 7.2.0, MI300

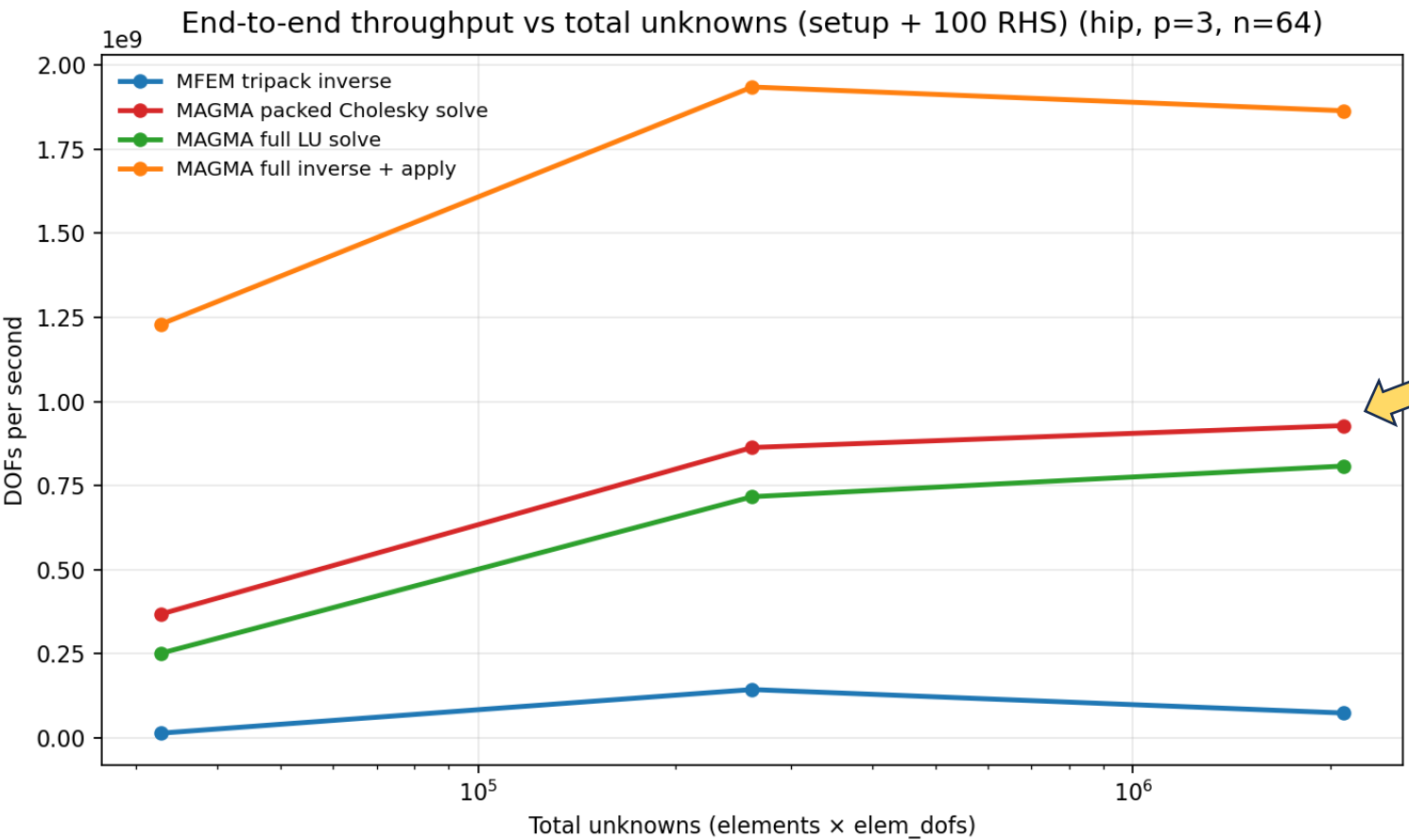
MAGMA end-to-end throughput



MAGMA packed inv

ROCM 7.2.0, MI300

MAGMA end-to-end throughput



MAGMA packed inverse routines for larger matrices are in progress....

MAGMA packed solve

ROCM 7.2.0, MI300

Packed triangular matrices clear winner for smaller size, further optimizations needed for larger matrices (n=64)

MFEM-mini-app: Matrix-Free Saddle-Point Solver

- Mixed Darcy equations:

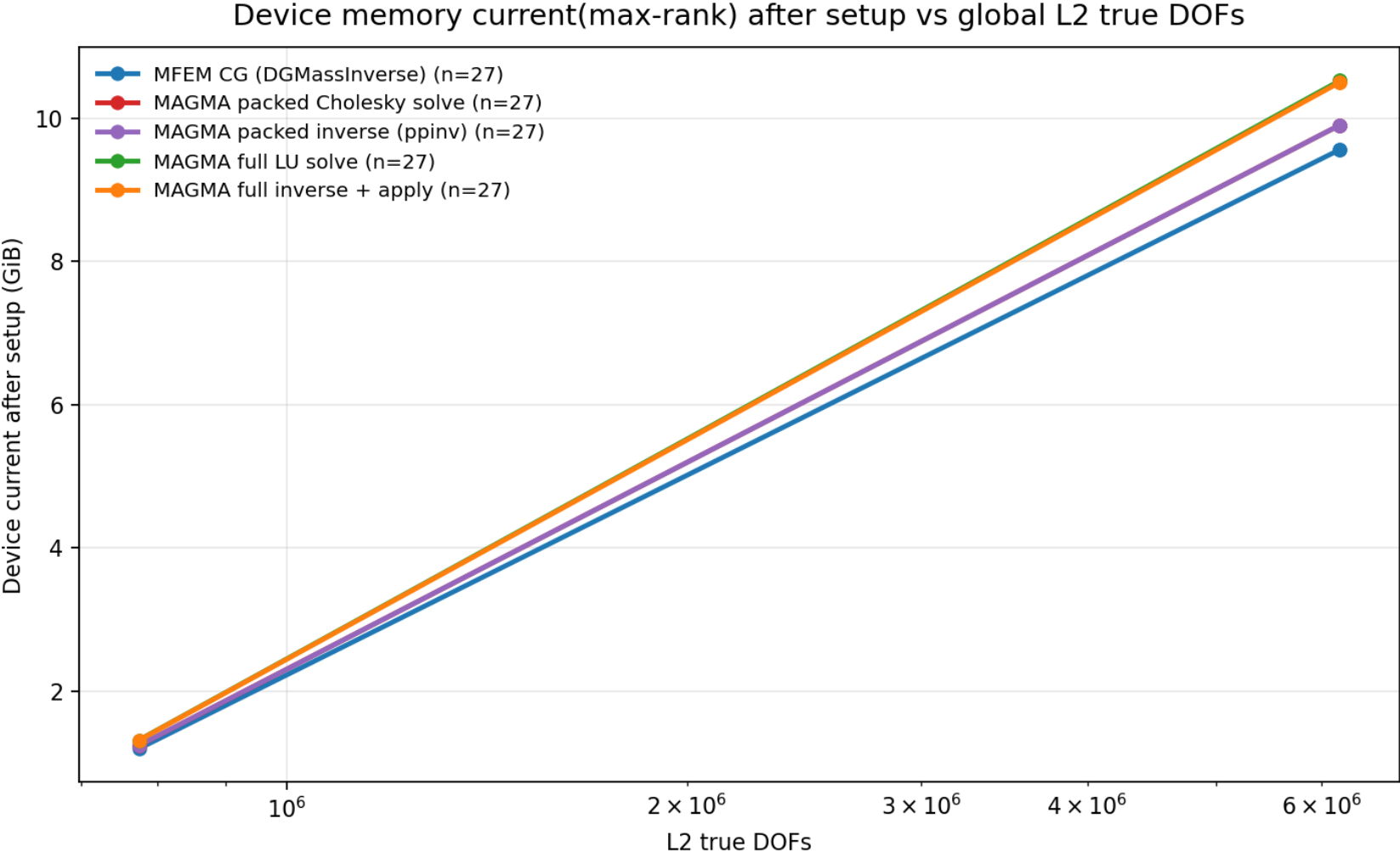
$$\begin{aligned}\kappa \mathbf{u} + \nabla p &= f, \\ -\nabla \cdot \mathbf{u} &= g\end{aligned}$$

- Natural boundary condition: $-p =$ given pressure
- **Discretization and solver**
 - Raviart–Thomas, H (div), finite elements for velocity u
 - Piecewise discontinuous polynomials for pressure p
- **Computational bottleneck**
 - Repeated element-local L^2 mass-matrix solves
 - Many small, dense, symmetric positive-definite systems
 - Independent local systems are well suited to batched accelerator kernels

Pazner, Will, Tzanio Kolev, and Panayot S. Vassilevski. "Matrix-free high-performance saddle-point solvers for high-order problems in." *SIAM Journal on Scientific Computing* 46.3 (2024): B179-B204.

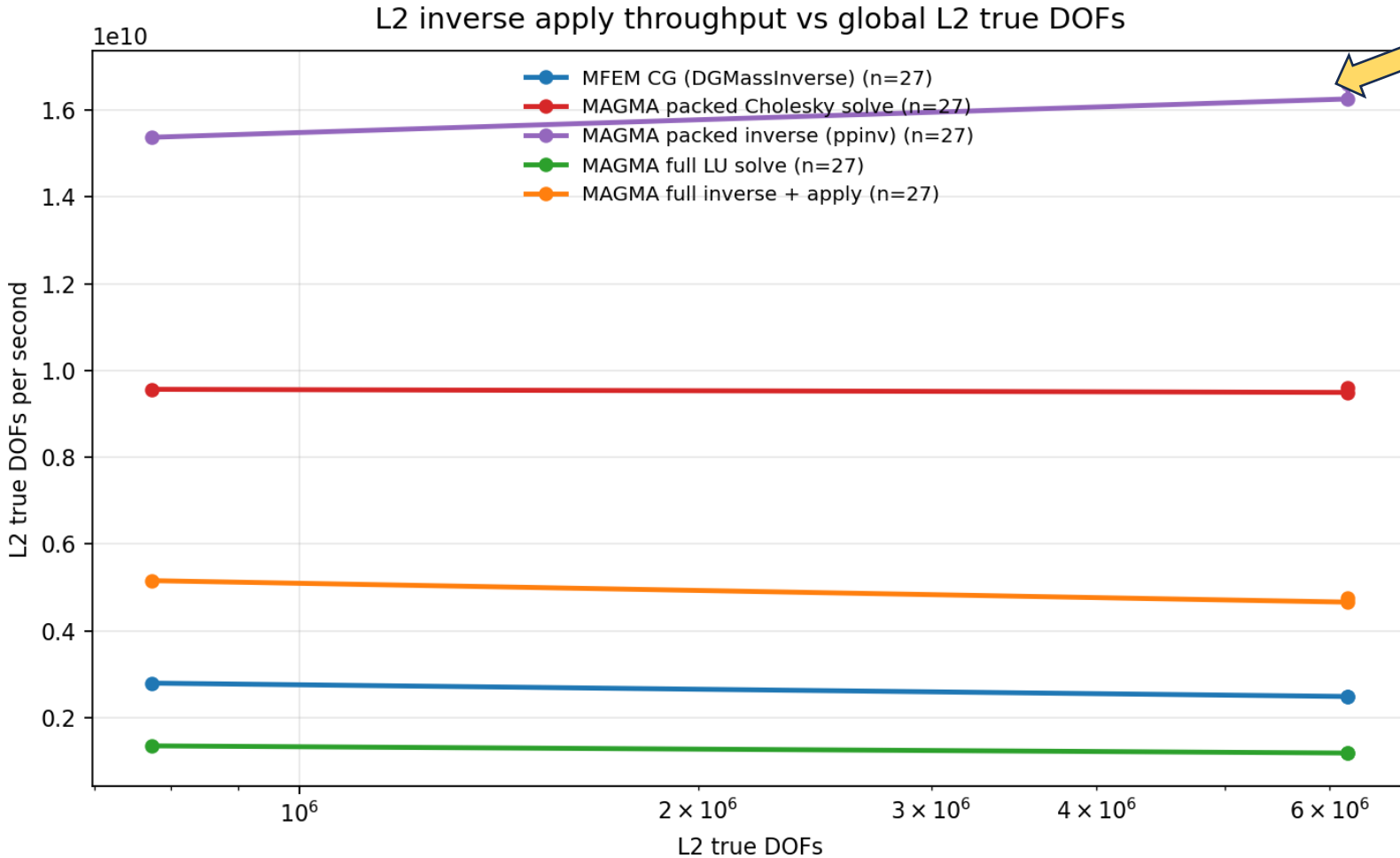
The saddle-point solver is scalable globally, but its repeated local mass solves are an important optimization target.

Saddle point solver memory



ROCM 7.2.0, MI300

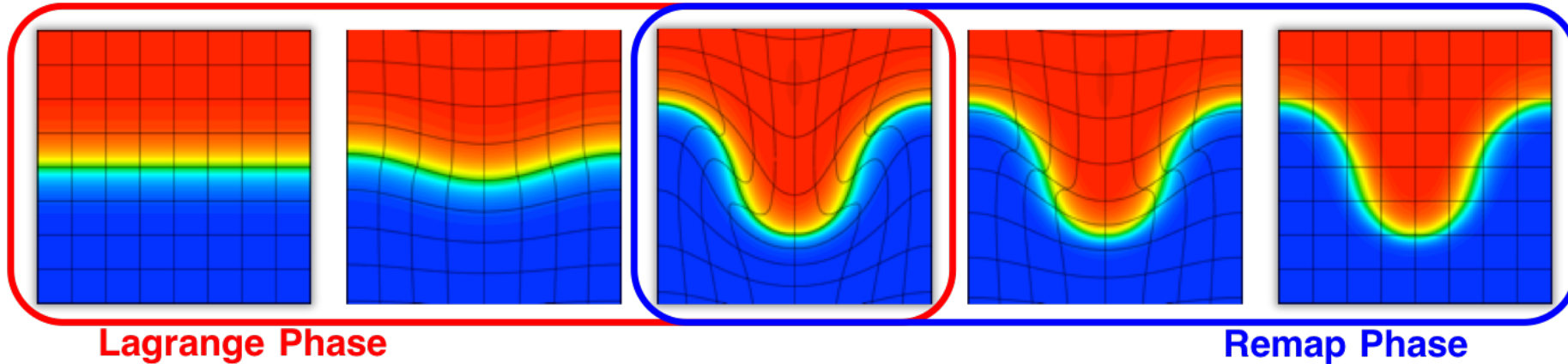
Saddle point solver throughput



MAGMA packed
inv

ROCM 7.2.0, MI300

MARBL is an arbitrary Lagrangian-Eulerian (ALE) rad-hydro code based on high-order FEM methods



- Evolution in a Lagrangian frame of reference
 - In this work we focus on quad/hex elements
- Mesh optimization according to a user specified objective function
- DG advection-based ALE remap transports field quantities to new mesh
 - Must be monotonic (i.e. no new extrema)
 - Conservative
 - Accurate (via high-order methods)

Local (L2) mass matrices are ubiquitous in MARBL

- DG quadrature $\leftrightarrow$ DoF projection / limiting (remap + transport-style workflows)
- Radiation diffusion (per-group block solves): repeatedly inverts element mass-like SPD blocks
- Thermal conduction (energy mass blocks in the conduction system)
- Turbulence (radiation-coupled pieces / similar mass-block pattern)

Integration in MARBL

- Incremental integration
 - We start with the DG Advection based ALE remap code
- We consider:
 - Total memory used
 - Cycle count / number of remap steps taken
 - Time to solution
- We use the Shaped Charged problem
 - 4 MI300's, 362,7520 quadrature points per device
 - Solve to $T=10$ (not the full run)

MARBL: Shaped charged benchmark

4 MI300's, 362,7520 qpts per device

Configuration	Memory pool requirement (GB)	Remap avg time/rank (s)	TimeStepLoop avg time/rank (s)	Total cycle count
MAGMA – full LU solve	54.04	381.61	998.99	3248
MAGMA packed-solve	51.87	320.68	900.93	3069
MAGMA packed-inverse	51.87	287.025	854.90	3062

Key Takeaways

- **Best overall:** MAGMA packed inv
- **TimeStepLoop speedup: 1.168x speedup**
- **MFRemap speedup: 1.330x speedup**

Conclusion

- Packed symmetric storage reduces local matrix memory footprint and data movement by approximately 2x.
- New MAGMA batched kernels enable performant GPU execution of packed Cholesky factorization, solves, inversion, and matrix-vector operations.
- MFEM integration allows element-local mass matrices to remain packed throughout assembly and local workflows.
- Packed kernels provide a practical path toward reducing memory pressure while maintaining GPU performance for small dense systems.